「ハナちゃん日記」投稿ガイド

Astro 製ブログの記事の書き方・公開のしかた

河合勝彦

名古屋市立大学大学院経済学研究科

kkawai@econ.nagoya-cu.ac.jp

2026 年 10 月 28 日

概要

本ガイドは、フレンチ・ブルドッグのハナちゃんの日記ブログ「ハナちゃん日記」に記事を追加・修正し、公開するまでの手順をまとめたものです。サイトは静的サイトジェネレーター Astro で作られ、GitHub Pages で公開しています。記事は Markdown ファイルで書き、GitHub にプッシュすると自動で公開されます。また、このリポジトリをコピーして自分のブログとして公開する方法も 10 節で説明します。

目次

1 サイトのしくみ 3
- 1.1 基本情報 . . . 3
- 1.2 公開までの流れ . . . 3
- 1.3 フォルダ構成 . . . 3

2 はじめての準備 4
- 2.1 必要なソフトウェア . . . 4
- 2.2 リポジトリの取得と初期設定 . . . 4

3 新しい記事を書く 4
- 3.1 ファイルを作る . . . 4
- 3.2 記事のひな形 . . . 5
- 3.3 フロントマターの項目 . . . 5
- 3.4 本文の書き方（Markdown） . . . 6

4 画像を使う 6
- 4.1 画像ファイルを置く . . . 6
- 4.2 記事の先頭画像（ヒーロー画像） . . . 6
- 4.3 本文中の画像 . . . 7

5 手元で確認する 7

6 公開する 7
6.1 コミットとプッシュ 7
6.2 公開の確認 8

7 記事を修正・削除する 8
7.1 修正 8
7.2 下書き 8
7.3 削除 8

8 GitHub の Web 画面だけで投稿する 8

9 サイト全体の設定を変える 9

10 このブログをもとに自分のブログを作る 9
10.1 全体の流れ 9
10.2 手順 1：GitHub アカウントと認証の準備 10
10.3 手順 2：クローンする 10
10.4 手順 3：GitHub に自分のリポジトリを作る 11
10.5 手順 4：プッシュ先を自分のリポジトリに切り替える 11
10.6 手順 5：設定を自分のものに書き換える 11
10.7 手順 6：手元で確認してプッシュする 13
10.8 手順 7：GitHub Pages を有効にして公開する 13

11 困ったときは 13

1　サイトのしくみ

1.1　基本情報

項目	内容
公開 URL	https://katzkawai.org/kklab-astro-blog/
リポジトリ	https://github.com/katzkawai/kklab-astro-blog
使用ツール	Astro 7（ブログテンプレートをもとに作成）
公開方法	GitHub Actions による自動ビルド → GitHub Pages

1.2　公開までの流れ

記事の公開は、次の流れで行います。

1. `src/content/blog/` に Markdown ファイル（`.md`）を作る
2. 手元のパソコンで表示を確認する（省略可）
3. Git でコミットし、GitHub の `main` ブランチへプッシュする
4. GitHub Actions が自動でサイトをビルドし、数分後に公開される

HTML を直接書く必要はありません。Markdown で本文を書けば、デザインの整ったページが自動で作られます。

1.3　フォルダ構成

記事の追加でさわるのは、主に次の 2 か所です。

```
kklab-astro-blog/
├── src/
│   ├── assets/          ← 画像を置く場所
│   ├── content/
│   │   └── blog/        ← 記事（Markdown）を置く場所
│   ├── components/      ← ヘッダー・フッターなどの部品
│   ├── layouts/         ← 記事ページのレイアウト
│   ├── pages/           ← トップ・一覧・プロフィールのページ
│   └── consts.ts        ← サイト名・著者名などの設定
├── public/              ← favicon など、そのまま公開するファイル
├── docs/guide/          ← このガイド
├── astro.config.mjs     ← Astro の設定（公開 URL など）
└── .github/workflows/   ← 自動公開の設定
```

2　はじめての準備

手元のパソコンで記事を書いて確認する場合は、最初に一度だけ次の準備をします（GitHub の Web 画面だけで投稿する場合は 8 節を参照）。

2.1　必要なソフトウェア

- **Git**：変更履歴の管理と GitHub へのアップロードに使います。
- **Node.js**（バージョン 22.23 以上、推奨は 24）：Astro を動かすのに使います。
- **テキストエディタ**：Visual Studio Code など。Markdown のプレビュー機能があると便利です。

2.2　リポジトリの取得と初期設定

ターミナルで次のコマンドを実行します。

```
git clone https://github.com/katzkawai/kklab-astro-blog.git
cd kklab-astro-blog
npm install
```

`npm install` は、Astro などの必要なパッケージをダウンロードします。初回は数分かかることがあります。

3　新しい記事を書く

3.1　ファイルを作る

`src/content/blog/` の中に、新しい Markdown ファイルを作ります。**ファイル名がそのまま記事の URL になる**ので、半角英数字とハイフンで名前をつけてください。

ファイル名	公開される URL
`hana-first-walk.md`	`.../kklab-astro-blog/blog/hana-first-walk/`
`2026-11-03-dogrun.md`	`.../kklab-astro-blog/blog/2026-11-03-dogrun/`

ファイル名のコツ

日本語やスペースは URL が読みにくくなるため避けましょう。`2026-11-03-dogrun.md` のように日付を先頭につけると、フォルダ内で並べたときに時系列順になって管理しやすくなります。

3.2　記事のひな形

ファイルの中身は、次のように書きます。先頭の `---` で囲まれた部分を**フロントマター**と呼び、記事のタイトルや日付などの情報を書きます。その下が本文です。

```
---
title: 'はじめてのドッグラン'
description: '近所のドッグランでお友だちがたくさんできました。'
pubDate: '2026-11-03'
heroImage: '../../assets/hana-dogrun.jpg'
tags: ['お出かけ', 'お友だち']
---

今日は、はじめてドッグランに行ってきました。

## 大きなわんちゃんにびっくり

入ってすぐ、ゴールデン・レトリーバーにあいさつされて……
```

3.3　フロントマターの項目

項目	必須	説明
`title`	○	記事のタイトル
`description`	○	一覧ページに表示される短い紹介文（1〜2 文）
`pubDate`	○	公開日。`'2026-11-03'` の形式で書く
`updatedDate`		更新日。記事を修正したときに追加すると「更新」と表示される
`heroImage`		記事の先頭とカードに表示する画像（4 節）
`tags`		タグ。`['お散歩', 'はじめて']` のように書く
`draft`		`true` にすると下書き扱いになり、公開されない

注意

フロントマターの書き方を間違えると、ビルドが失敗して公開されません。特に次の点に気をつけてください。

- 先頭と末尾の `---` は、半角ハイフン 3 つ
- `title:` のコロンのあとには半角スペース
- 値は半角の `'`（シングルクォート）で囲む
- 日付は `YYYY-MM-DD` 形式

3.4　本文の書き方（Markdown）

本文では、次のような Markdown の記法が使えます。

書き方	表示
`## 見出し`	中見出し
`### 小見出し`	小見出し
`**太字**`	**太字**
`- 項目`	箇条書き
`1. 項目`	番号付きリスト
`> 引用文`	引用
`[文字](https://...)`	リンク
`![説明](../../assets/xx.jpg)`	画像

表は次のように書きます。

```
| 項目 | 記録 |
| --- | --- |
| 歩いた時間 | 約 15 分 |
| 歩いた距離 | 約 400 メートル |
```

段落と改行

段落を分けるときは、空行を 1 行入れます。日本語の文章を途中で改行すると、表示したときに余分な空白が入ることがあります。**1 つの段落は改行せずに 1 行で書く**のがおすすめです。

4　画像を使う

4.1　画像ファイルを置く

写真は `src/assets/` フォルダに置きます。ファイル名は半角英数字にしてください（例：`hana-dogrun.jpg`）。JPEG・PNG・WebP が使えます。

Astro はビルド時に画像を自動で縮小・WebP 形式に変換するので、スマートフォンで撮った大きな写真をそのまま置いてもかまいません。ただし、リポジトリが重くならないよう、目安として 1 枚 5MB 以下にしておくとよいでしょう。

4.2　記事の先頭画像（ヒーロー画像）

フロントマターの `heroImage` に、記事ファイルから見た画像の場所を書きます。記事は `src/content/blog/` にあるので、`../../assets/` で `src/assets/` を指します。

```
heroImage: '../../assets/hana-dogrun.jpg'
```

横長（縦横比 2:1）の画像が最もきれいに表示されます。

4.3　本文中の画像

本文の好きな位置に、次のように書きます。[ ] の中には画像の説明（代替テキスト）を書きます。

```
![芝生で走るハナちゃん](../../assets/hana-dogrun-2.jpg)
```

プライバシーへの配慮

写真に自宅の外観や住所がわかるもの、他の飼い主さんの顔などが写っていないか、公開前に確認しましょう。スマートフォンの写真には撮影場所（位置情報）が記録されていることがあります。元の画像ファイルもそのまま公開されることがあるので、カメラの位置情報をオフにして撮影するか、アップロード前に位置情報を削除してください。

5　手元で確認する

公開する前に、自分のパソコンで表示を確認できます。

```
npm run dev
```

ターミナルに表示されるアドレス（通常は http://localhost:4321/kklab-astro-blog/）をブラウザで開きます。開発サーバーを起動したままファイルを保存すると、ブラウザの表示が自動で更新されます。終了するときはターミナルで Ctrl + C を押します。

本番と同じ形でビルドできるか確かめたい場合は、次のコマンドを使います。エラーが出なければ、公開してもビルドは成功します。

```
npm run build
npm run preview
```

6　公開する

6.1　コミットとプッシュ

記事ができたら、次のコマンドで GitHub に送ります。

```
git add src/content/blog/2026-11-03-dogrun.md src/assets/hana-dogrun.jpg
git commit -m "記事を追加：はじめてのドッグラン"
```

```
git push
```

6.2　公開の確認

プッシュすると、GitHub Actions が自動でビルドと公開を始めます。

1. GitHub のリポジトリページで「Actions」タブを開く
2. 一番上の実行（ワークフロー名「Deploy to GitHub Pages」）が緑色のチェックになれば成功
3. 2〜3 分後に公開 URL を開いて、記事が表示されることを確認する

赤い × 印になった場合はビルドに失敗しています。実行をクリックするとエラーの内容が確認できます（11 節）。

7　記事を修正・削除する

7.1　修正

Markdown ファイルを編集してプッシュすれば、その内容で公開し直されます。内容を大きく変えたときは、フロントマターに `updatedDate` を追加すると、記事に更新日が表示されます。

```
pubDate: '2026-11-03'
updatedDate: '2026-11-05'
```

7.2　下書き

書きかけの記事は、フロントマターに `draft: true` を書いておくと、プッシュしても公開されません。公開するときは、この行を消すか `false` にします。なお、下書きでもリポジトリ（GitHub）上ではファイルの中身が見える点に注意してください。

7.3　削除

記事のファイルを削除してプッシュします。

```
git rm src/content/blog/2026-11-03-dogrun.md
git commit -m "記事を削除：はじめてのドッグラン"
git push
```

8　GitHub の Web 画面だけで投稿する

パソコンに Git や Node.js を入れなくても、ブラウザだけで投稿できます。

1. https://github.com/katzkawai/kklab-astro-blog を開き、src/assets フォルダへ移動する
2.「Add file」→「Upload files」で写真をアップロードし、「Commit changes」を押す
3. src/content/blog フォルダへ移動し、「Add file」→「Create new file」を選ぶ
4. ファイル名（例：2026-11-03-dogrun.md）を入力し、フロントマターと本文を書く
5.「Commit changes」を押すと、自動でビルドと公開が始まる

既存の記事を直すときは、ファイルを開いて右上の鉛筆アイコン（Edit）から編集できます。

9　サイト全体の設定を変える

変えたいもの	編集するファイル
サイト名・説明文・著者名・連絡先	src/consts.ts
「ハナちゃんについて」のプロフィール	src/pages/about.astro
トップページの文章・画像	src/pages/index.astro
色・文字の大きさ	src/styles/global.css
ヘッダーのメニュー	src/components/Header.astro
タブのアイコン	public/favicon.svg
公開 URL	astro.config.mjs の site と base

たとえばプロフィールの「好きなもの」を変えるには、src/pages/about.astro の該当する行を書き換えます。

```
<tr><th>好きなもの</th><td>ささみ、ひなたぼっこ、ピンクのぬいぐるみ</td></tr>
```

10　このブログをもとに自分のブログを作る

このリポジトリをまるごとコピーして、自分の GitHub アカウントで「自分のブログ」として公開する手順です。ハナちゃんの記事や写真を自分のものに置き換えれば、デザインや自動公開のしくみはそのまま使えます。

10.1　全体の流れ

1. GitHub のアカウントと、プッシュのための認証を用意する
2. このリポジトリを手元のパソコンにクローン（コピー）する
3. GitHub に自分用の空のリポジトリを作る
4. プッシュ先（送り先）を自分のリポジトリに切り替える
5. 公開 URL やサイト名などの設定を自分のものに書き換える
6. 手元で確認して、自分のリポジトリへプッシュする
7. GitHub Pages を有効にして、公開を確認する

以下では、GitHub のユーザー名を **<ユーザー名>**、新しく作るリポジトリの名前を **<リポジトリ名>** と書きます。自分のものに置き換えて読んでください（< と > は入力しません）。

「Fork」との違い

GitHub のリポジトリページにある「Fork」ボタンでもコピーは作れます。ただしフォークしたリポジトリでは GitHub Actions が最初は止まっていて手動で有効にする必要があるなど、初めての方にはわかりにくい点があります。本ガイドでは、クローンして新しいリポジトリへプッシュする、いちばん見通しのよい方法を説明します。

10.2　手順 1：GitHub アカウントと認証の準備

- GitHub のアカウントを持っていなければ、https://github.com/ で無料で作成します。ユーザー名は公開 URL の一部になるので、半角英数字で読みやすいものにしましょう。
- Git と Node.js をインストールしておきます（2 節）。
- パソコンから GitHub へプッシュするには「認証」が必要です。いちばん手軽なのは GitHub CLI（`gh`）を https://cli.github.com/ からインストールし、ターミナルで次を実行する方法です。

```
gh auth login
```

質問には「GitHub.com」「HTTPS」「Yes」「Login with a web browser」の順に答え、表示された 8 桁のコードをブラウザに入力すればログインが完了します。この設定は一度行えば、以降の `git push` でそのまま使われます。

パスワードを聞かれたら

`gh` を使わない場合、`git push` のときにユーザー名とパスワードを聞かれることがあります。ここで入力するのは GitHub にログインするときのパスワードではなく、GitHub の「Settings」→「Developer settings」→「Personal access tokens」で作る**パーソナルアクセストークン**です。ログイン用のパスワードを入れても拒否されるので注意してください。

10.3　手順 2：クローンする

ターミナル（Windows なら Git Bash か PowerShell、Mac なら「ターミナル」）を開き、ブログを置きたいフォルダに移動してから次を実行します。

```
git clone https://github.com/katzkawai/kklab-astro-blog.git <リポジトリ名>
cd <リポジトリ名>
npm install
```

1 行目の最後の **<リポジトリ名>** は、手元に作るフォルダの名前です。手順 3 で作るリポジトリと

同じ名前にしておくとわかりやすいでしょう（省略すると `kklab-astro-blog` という名前になります）。すでに 2 節の手順でクローンしてある場合は、そのフォルダをそのまま使ってかまいません。

10.4　手順 3：GitHub に自分のリポジトリを作る

1. GitHub にログインし、右上の「＋」→「**New repository**」を選ぶ
2. 「Repository name」に `<リポジトリ名>` を入力する。半角英数字とハイフンだけを使う（例：`my-dog-blog`）
3. 「**Public**」を選ぶ（無料プランで GitHub Pages を使うには Public が必要）
4. 「Add a README file」「Add .gitignore」「Choose a license」は**すべてチェックを外したまま**にする（中身が空のリポジトリにする）
5. 「**Create repository**」を押す

作成後に表示される画面（コマンドの説明が並んだページ）はそのままにして、次の手順に進みます。

リポジトリ名が URL になる

公開 URL は `https://<ユーザー名>.github.io/<リポジトリ名>/` になります。たとえばユーザー名が `taro`、リポジトリ名が `my-dog-blog` なら、`https://taro.github.io/my-dog-blog/` です。あとから変えることもできますが、そのときは手順 5 の設定も合わせて直す必要があります。

10.5　手順 4：プッシュ先を自分のリポジトリに切り替える

クローンした直後は、プッシュ先（Git では `origin` と呼びます）がもとのハナちゃん日記のリポジトリになっています。これを自分のリポジトリに切り替えます。

```
git remote set-url origin https://github.com/<ユーザー名>/<リポジトリ名>.git
git remote -v
```

2 行目の結果が次のように自分のユーザー名とリポジトリ名になっていれば、切り替え完了です。

```
origin  https://github.com/<ユーザー名>/<リポジトリ名>.git (fetch)
origin  https://github.com/<ユーザー名>/<リポジトリ名>.git (push)
```

10.6　手順 5：設定を自分のものに書き換える

テキストエディタで次の 2 つのファイルを開き、書き換えます。

■(a) 公開 URL：`astro.config.mjs`　`site` と `base` の 2 行を、自分のユーザー名とリポジトリ名に合わせます。

```
site: 'https://<ユーザー名>.github.io',
base: '/<リポジトリ名>',
```

	ハナちゃん日記（現在）	例：taro さんの my-dog-blog
site	'https://katzkawai.org'	'https://taro.github.io'
base	'/kklab-astro-blog'	'/my-dog-blog'
公開 URL	https://katzkawai.org/kklab-astro-blog/	https://taro.github.io/my-dog-blog/

ここを間違えると表示がくずれる

base がリポジトリ名と合っていないと、ページは開けても CSS や画像が読み込まれず、まっ白な表示になります。先頭のスラッシュ（/）を忘れずに、大文字・小文字もリポジトリ名と正確に合わせてください。なお、リポジトリ名を <ユーザー名>.github.io にした場合だけは、公開 URL が https://<ユーザー名>.github.io/ になるので、base の行を削除します。独自ドメインを使う場合は site をそのドメインにします（ハナちゃん日記では katzkawai.org を使っています）。

■(b) サイト名・著者名：src/consts.ts　サイト名、説明文、著者名、所属、連絡先を自分のものにします。

```
export const SITE_TITLE = 'ポチの日記';
export const SITE_DESCRIPTION = '柴犬のポチ（2 歳・男の子）の毎日をつづる日記です。';
export const SITE_AUTHOR = '山田太郎';
export const SITE_AFFILIATION = '東京都';
export const SITE_CONTACT = 'taro@example.com';
```

SITE_AUTHOR・SITE_AFFILIATION・SITE_CONTACT は、フッターと「〜について」ページに表示されます。公開したくない項目は ''（空）にするか、src/components/Footer.astro と src/pages/about.astro の該当行を削除してください。

■(c) あとで直せばよいもの　トップページの文章（src/pages/index.astro）、プロフィール（src/pages/about.astro）、画像（src/assets/）、タブのアイコン（public/favicon.svg）は、9 節を参考に、公開してから少しずつ自分のものに置き換えていけば十分です。

サンプル記事の消し方

ハナちゃんの記事 3 本は src/content/blog/ にあります。自分の記事を 1 本書いて表示を確認したあとで、git rm src/content/blog/hana-*.md のように削除すると安心です。ただし画像 src/assets/hana-about.jpg はトップページとプロフィールページでも使っているので、別の画像に差し替えるまで消さないでください（消すとビルドが失敗します）。

10.7　手順 6：手元で確認してプッシュする

まず手元で表示を確認します（5 節）。アドレスの末尾は、自分のリポジトリ名になっています。

```
npm run dev
```

`http://localhost:4321/<リポジトリ名>/` をブラウザで開き、サイト名などが自分のものに変わっていれば OK です。`Ctrl + C` で止め、変更をコミットして自分のリポジトリへプッシュします。

```
git add -A
git commit -m "自分のブログ用に設定を変更"
git push -u origin main
```

`-u origin main` をつけるのは初回だけで、2 回目以降は `git push` だけでかまいません。ブラウザで自分のリポジトリページを再読み込みし、ファイル一覧が表示されればプッシュ成功です。

10.8　手順 7：GitHub Pages を有効にして公開する

1. 自分のリポジトリページで「**Settings**」タブを開き、左のメニューから「**Pages**」を選ぶ
2. 「Build and deployment」の「Source」を「**GitHub Actions**」に変える
3. 「**Actions**」タブを開く。手順 6 のプッシュで始まった「Deploy to GitHub Pages」が赤い × で終わっていたら、左の一覧から「Deploy to GitHub Pages」を選び、右側の「**Run workflow**」→「Run workflow」で実行し直す
4. 緑色のチェックになったら、`https://<ユーザー名>.github.io/<リポジトリ名>/` を開く

自分のブログが表示されれば完成です。以降の記事の書き方・公開のしかたは、3 節から 6 節までと同じです。もとのリポジトリ（ハナちゃん日記）とはつながっていないので、ハナちゃん日記側の更新を気にする必要はありません。

最初のプッシュで Actions が失敗するのはふつうです

GitHub Pages を有効にする前にプッシュすると、ビルドは成功しても最後の公開（deploy）の段階で失敗します。これは設定の順番によるもので、記事や設定の間違いではありません。Pages を有効にしてから「Run workflow」で実行し直せば成功します。

11　困ったときは

Actions が赤い × になった

多くはフロントマターの書き間違いか、画像のパス違いです。エラーメッセージに出ているファイル名と行を確認してください。`heroImage` の画像が `src/assets/` に存在するか、ファイル名の

大文字・小文字が一致しているかも確かめます。

記事が一覧に出てこない

`draft: true` になっていないか、ファイルの拡張子が `.md` になっているか、`src/content/blog/` の中に置いたかを確認します。

記事の順番がおかしい

一覧は `pubDate` の新しい順に並びます。日付の書き方を確認してください。

プッシュしたのに表示が変わらない

ビルドの完了を待ってから、ブラウザを再読み込み（キャッシュを無視するには `Ctrl + Shift + R`）してください。

`git push` が拒否された

GitHub 上で先に別の変更（Web 画面での編集など）がされています。`git pull` を実行してからもう一度 `git push` します。

本ガイドに関する問い合わせ先：河合勝彦（名古屋市立大学大学院経済学研究科）
kkawai@econ.nagoya-cu.ac.jp